# STAT 612: Week 6: Midterm 1 (40 pts)

*Your Name*

*2019-10-10*

## Instructions

- Please place your answers in an R Markdown document.
- Submit this R Markdown document, and the resulting knitted PDF, to Blackboard before the exam time has expired.
- **Late submissions will have 1/2 point deducted for each minute past the due time.**
- Make sure all code is visible in the PDF by making sure `knitr::opts_chunk$set(echo = TRUE)` is in your first code chunk.
- Knit early and knit often. It is your responsibility to make sure your code knits without errors. Do not wait to knit until the very end. Submitting an Rmd that does not knit to PDF will result in point deductions.
- You have two-hours and thirty minutes.
- You may use your notes, the R help files, and the internet.
- You may **not** discuss this exam with anyone (including over instant message or text).
- Use the tidyverse, not base R, where possible. Too much base R will result in point deductions.

## Question 1: Housing Sales in Texas

In this exercise, we will use the `txhousing` data frame from the ggplot2 package. These data contain information on the housing market in various cities in Texas for each month from 2000 to 2015. Variables include:

- `city`: The name of the multiple listing system area.
- `year`: The year.
- `month`: The month.
- `date`: The date.
- `sales`: The total number of sales.
- `volume`: The total value of sales.
- `median`: The median sale price.
- `listings`: The total number of active listings (number of houses up for sale).
- `inventory`: The amount of time it would take to sell all current listings at the current pace of sales.

1. (2 pts) Load the package(s) necessary to complete this midterm and load in the `txhousing` data.

```
library(tidyverse)
data("txhousing")
```

2. (2 pts) Use appropriate R code to find the 1095th element of the `median` variable from the `txhousing` data frame.

```
txhousing$median[1095]
```

```
## [1] 131400
```

3. (4 pts) Use appropriate R code to determine how many different cities are in the dataset.

```
## One way
length(unique(txhousing$city))
```

```
## [1] 46
```

```
## Another way
n_distinct(txhousing$city)
```

```
## [1] 46
```

4. (4 pts) Use the tidyverse to calculate

   i. The total number of sales in each year in each city and
   ii. The total volume of sales in each year in each city.

   Save this output as a new data frame.

```
txhousing %>%
  group_by(city, year) %>%
  summarize(tot_sales = sum(sales, na.rm = TRUE),
            tot_vol = sum(volume, na.rm = TRUE)) ->
  sumdf
```

5. (5 pts) Use the tidyverse to add two new variables to `txhousing`:

   i. `ave_price`: the average sales price, and
   ii. `prop_sale`: the proportion of listings that were sold.

```
txhousing %>%
  mutate(ave_price = volume / sales,
         prop_salle = sales / listings) ->
  txhousing
```

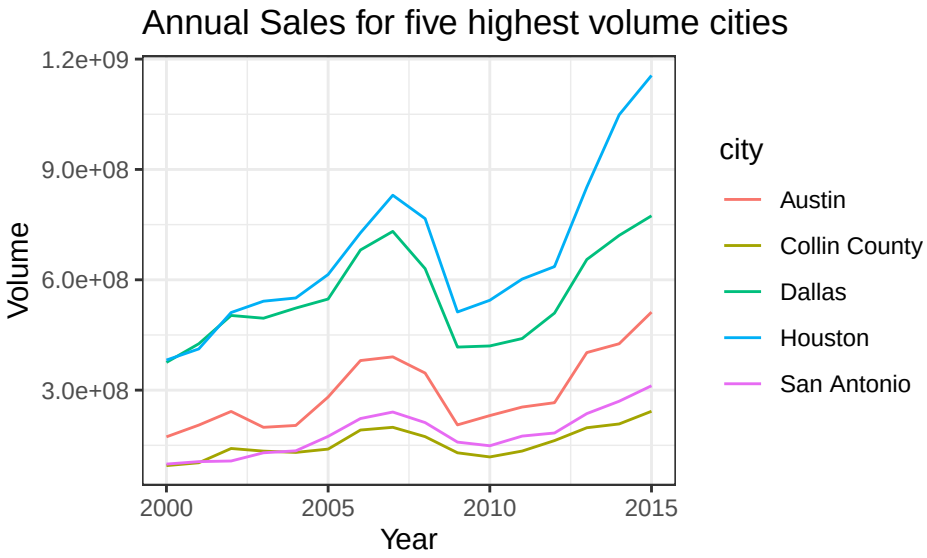
6. (5 pts) Use the tidyverse to find the five cities with the largest total volume of sales for 2015. You should get `Houston`, `Dallas`, `Austin`, `San Antonio`, and `Collin County`.

```
txhousing %>%
  filter(year == 2015) %>%
  group_by(city) %>%
  summarize(totval = sum(volume, na.rm = TRUE)) %>%
  arrange(desc(totval)) %>%
  head(n = 5)
```

```
## # A tibble: 5 x 2
##   city               totval
##   <chr>               <dbl>
## 1 Houston       13380209058
## 2 Dallas        10533452560
## 3 Austin         6188757237
## 4 San Antonio    3796213948
## 5 Collin County  3186815772
```

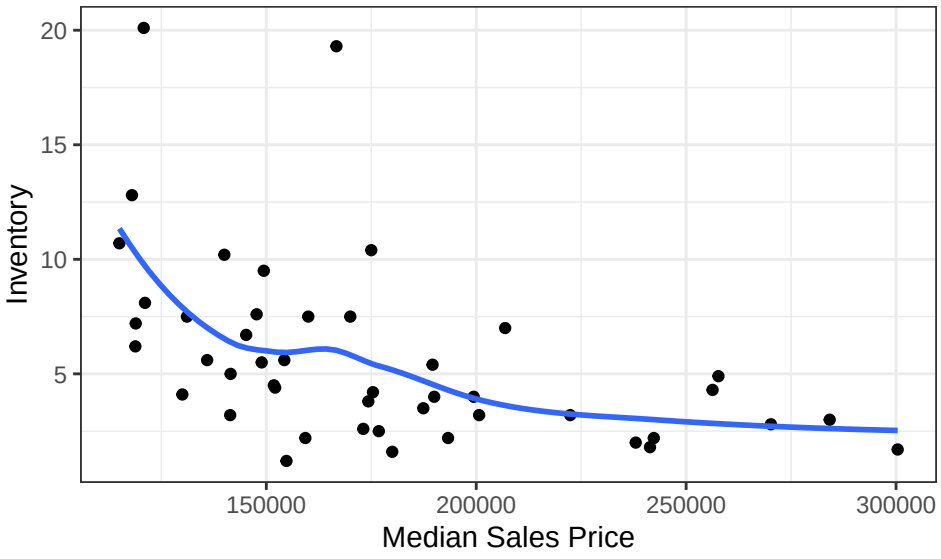
7. (5 pts) Use the tidyverse to select the rows that contain data from January from the five cities from part 6. Use this new data frame to create the following plot.

```
txhousing %>%
  filter(city %in% c("Houston", "Dallas", "Austin", "San Antonio", "Collin County"),
         month == 1) %>%
  ggplot(aes(x = year, y = volume, color = city)) +
  geom_line() +
  theme_bw() +
  xlab("Year") +
  ylab("Volume") +
  ggtitle("Annual Sales for five highest volume cities")
```




8. (5 pts) Use the tidyverse to select all rows from June, 2015. Use these data to make an appropriate plot to explore the relationship between median sales price and inventory. Make your plot pretty. Does the relationship appear to be positively or negatively associated?

```
txhousing %>%
  filter(year == 2015, month == 6) %>%
  ggplot(aes(x = median, y = inventory)) +
  geom_point() +
  geom_smooth(se = FALSE) +
  theme_bw() +
  xlab("Median Sales Price") +
  ylab("Inventory")
```




The relationship appears to be negatively associated.

## Question 2: Coding Exercises

1. (4 pts) Reproduce the following with markdown:

   - **The**
   - `quick`

   1. ~~brown~~
   2. fox

      i. jumps
      ii. over

| | |
|---|---|
| the | lazy |
| dog | ! |

2. (2 pts) Create a function `add_odd()` that takes as input two integers and calculates the sum of all odd numbers that are both less than 100 and in the closed interval bounded by the two numbers. Use comments. The `min()`, `max()`, and `seq()` functions might be useful here. For example, here is some output:

```
add_odd <- function(a, b) {
  seqvec <- seq(min(a, b), max(a, b))
  logic_vec <- (seqvec %% 2 == 1) & (seqvec < 100)
  sumval <- sum(seqvec[logic_vec])
  return(sumval)
}
```

```
add_odd(1, 3) ## 1 + 3
```

```
## [1] 4
```

```r
add_odd(7, 2) ## 3 + 5 + 7
```

```
## [1] 15
```

```r
add_odd(97, 103) ## 97 + 99. The numbers 101 and 103 are not included
```

```
## [1] 196
```

Evaluate your function at the following inputs:

```r
add_odd(50, 150)
```

```
## [1] 1875
```

```r
add_odd(22, 1)
```

```
## [1] 121
```

```r
add_odd(-1, 1)
```

```
## [1] 0
```

3. (2 pts) Create a function, `rep_ave()` that takes two inputs: `x`, a vector of arbitrary length of numerics and `(ii)i, an integer between 1 and the length of the vectorx. The default fori‘` should be 1.

The function replaces the `i`th element of `x` with the mean of every element of `x` except the `i`th element of `x`. It then returns the modified vector. Here are some example outputs:

```r
rep_ave <- function(x, i = 1) {
  if (i < 1 | i > length(x)) {
    stop("i needs to be between 1 and length(x)")
  }

  mean_val <- mean(x[-i])
  x[i] <- mean_val
  return(x)
}
```

```r
  x <- c(1, 3, 5)
  rep_ave(x = x, i = 1) # First element is the average of 3 and 5
```

```
## [1] 4 3 5
```

```r
  rep_ave(x = x) # default for i is 1
```

```
## [1] 4 3 5
```

```r
  rep_ave(x = x, i = 2) # Second element is average of 1 and 5
```

```
## [1] 1 3 5
```

```r
  rep_ave(x = x, i = 3) # Third element is average of 1 and 3
```

```
## [1] 1 3 2
```

```r
  x <- c(9, 0, 0, 1000)
  rep_ave(x, 4) # Fourth element is average of 9, 0, and 0.
```

```
## [1] 9 0 0 3
```

Evaluate your function at the following inputs:

```r
  x <- c(100, 0, 9, 44, 9, 100, 100, 2)
  rep_ave(x, 1)
```

```
## [1]  37.71429   0.00000   9.00000  44.00000   9.00000 100.00000 100.00000
## [8]   2.00000
```

```r
  rep_ave(x)
```

```
## [1]  37.71429   0.00000   9.00000  44.00000   9.00000 100.00000 100.00000
## [8]   2.00000
```

```r
  rep_ave(x, 4)
```

```
## [1] 100.00000   0.00000   9.00000  45.71429   9.00000 100.00000 100.00000
## [8]   2.00000
```